

Dynamic Programming

Basic to Advanced — Patterns, Solutions & Dry Runs

A structured guide covering the most common DP interview patterns, with recurrence relations, Python code, and step-by-step dry runs.

Table of Contents

- 1. Introduction to Dynamic Programming
- 2. Pattern 1 — Linear / 1D DP (Fibonacci, Climbing Stairs, House Robber)
- 3. Pattern 2 — 2D Grid DP (Unique Paths, Minimum Path Sum)
- 4. Pattern 3 — Knapsack Family (0/1 Knapsack, Subset Sum, Coin Change)
- 5. Pattern 4 — Longest Increasing Subsequence (LIS)
- 6. Pattern 5 — String DP (LCS, Edit Distance, Palindromic Substrings)
- 7. Pattern 6 — Interval / Partition DP (Matrix Chain Multiplication, Burst Balloons)
- 8. Advanced — Bitmask DP (Traveling Salesman Problem)
- 9. Advanced — DP on Trees (House Robber III)
- 10. How to Recognize Which Pattern to Use (Cheat Sheet)

1. Introduction to Dynamic Programming

What is DP?

Dynamic Programming (DP) is a technique for solving problems by breaking them into overlapping subproblems, solving each subproblem once, and storing (caching) the result so it is never recomputed. It applies when a problem has:

1. **Optimal Substructure** — the optimal solution to the problem can be built from optimal solutions to its subproblems.

2. **Overlapping Subproblems** — the same subproblems are solved again and again in a naive recursive solution.

Two Ways to Implement DP

Approach	How it works	Pros	Cons
Memoization (Top-Down)	Write the normal recursion, cache results in a map/array	Easy to derive from brute force	Recursion overhead, stack depth limits
Tabulation (Bottom-Up)	Build a table iteratively from base cases upward	No recursion overhead, often allows space optimization	Order of filling table must be worked out

General Steps to Solve Any DP Problem

1. **Identify state:** what parameters uniquely define a subproblem? (e.g. index, remaining capacity)

2. **Define dp[state]:** in plain English, what does this value represent?

3. **Find the recurrence:** how does dp[state] relate to smaller states?

4. **Base cases:** smallest subproblems with known answers.

5. **Order of computation:** make sure dependencies are solved first.

6. **Optimize space** if only the last few states are needed.

2. Pattern 1 — Linear / 1D DP

PATTERN: $dp[i]$ depends on $dp[i-1]$, $dp[i-2]$, ...

2.1 Fibonacci Numbers

Problem: Find the n -th Fibonacci number, where $F(0)=0$, $F(1)=1$, $F(n)=F(n-1)+F(n-2)$.

State: $dp[i]$ = the i -th Fibonacci number.

Recurrence: $dp[i] = dp[i-1] + dp[i-2]$, base cases $dp[0]=0$, $dp[1]=1$.

```
def fib(n):
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

Dry Run: fib(6)

i	0	1	2	3	4	5	6
dp[i]	0	1	1	2	3	5	8

$dp[2]=dp[1]+dp[0]=1$, $dp[3]=dp[2]+dp[1]=2$, $dp[4]=dp[3]+dp[2]=3$, $dp[5]=dp[4]+dp[3]=5$, $dp[6]=dp[5]+dp[4]=8$ → **Answer: 8**

Time: $O(n)$ | Space: $O(n)$, optimizable to $O(1)$ using two variables.

2.2 Climbing Stairs

Problem: You can climb 1 or 2 steps at a time. How many distinct ways to reach step n ?

State: $dp[i]$ = number of ways to reach step i .

Recurrence: $dp[i] = dp[i-1] + dp[i-2]$, base: $dp[0]=1$, $dp[1]=1$.

```
def climbStairs(n):
    if n <= 1:
        return 1
    prev2, prev1 = 1, 1
    for i in range(2, n + 1):
        cur = prev1 + prev2
        prev2, prev1 = prev1, cur
    return prev1
```

Dry Run: n = 5

i	0	1	2	3	4	5
dp[i]	1	1	2	3	5	8

Same recurrence shape as Fibonacci → **Answer: 8 ways**

2.3 House Robber

Problem: Given house values, rob houses to maximize total money without robbing two adjacent houses.

State: $dp[i]$ = max money robbing from houses $[0..i]$.

Recurrence: $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$

```
def rob(nums):
    if not nums:
        return 0
    prev2, prev1 = 0, 0
    for num in nums:
        cur = max(prev1, prev2 + num)
        prev2, prev1 = prev1, cur
    return prev1
```

Dry Run: nums = [2, 7, 9, 3, 1]

i	house value	$dp[i] = \max(dp[i-1], dp[i-2]+val)$
0	2	2

1	7	$\max(2, 0+7) = 7$
2	9	$\max(7, 2+9) = 11$
3	3	$\max(11, 7+3) = 11$
4	1	$\max(11, 11+1) = 12$

Answer: 12 (rob houses with value 2, 9, 1 $\rightarrow$ 2+9+1=12)

3. Pattern 2 — 2D Grid DP

PATTERN: $dp[i][j]$ depends on $dp[i-1][j]$ and $dp[i][j-1]$

3.1 Unique Paths

Problem: A robot at top-left of an $m \times n$ grid can move only right or down. Count paths to bottom-right.

State: $dp[i][j]$ = number of ways to reach cell (i,j) .

Recurrence: $dp[i][j] = dp[i-1][j] + dp[i][j-1]$; first row/column = 1.

```
def uniquePaths(m, n):
    dp = [[1] * n for _ in range(m)]
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i-1][j] + dp[i][j-1]
    return dp[m-1][n-1]
```

Dry Run: $m = 3, n = 3$

	j=0	j=1	j=2
i=0	1	1	1
i=1	1	2	3
i=2	1	3	6

$dp[1][1]=dp[0][1]+dp[1][0]=1+1=2$; $dp[2][2]=dp[1][2]+dp[2][1]=3+3=6 \rightarrow$ **Answer: 6**

3.2 Minimum Path Sum

Problem: Grid of non-negative numbers; find path from top-left to bottom-right minimizing sum, moving only right/down.

Recurrence: $dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$

```
def minPathSum(grid):
    m, n = len(grid), len(grid[0])
    dp = [[0] * n for _ in range(m)]
    dp[0][0] = grid[0][0]
    for j in range(1, n):
        dp[0][j] = dp[0][j-1] + grid[0][j]
    for i in range(1, m):
        dp[i][0] = dp[i-1][0] + grid[i][0]
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])
    return dp[m-1][n-1]
```

Dry Run: grid = $[[1,3,1],[1,5,1],[4,2,1]]$

	j=0	j=1	j=2
i=0	1	4	5
i=1	2	7	6
i=2	6	8	7

Path $1 \rightarrow 3 \rightarrow 1 \rightarrow 1$ gives sum 7 $\rightarrow$ **Answer: 7**

4. Pattern 3 — Knapsack Family

PATTERN: $dp[i][cap]$ — choose to include/exclude item i

4.1 0/1 Knapsack

Problem: Given weights and values of n items and capacity W , maximize value without exceeding capacity; each item used at most once.

State: $dp[i][w]$ = max value using first i items with capacity w .

Recurrence: $dp[i][w] = \max(dp[i-1][w], dp[i-1][w-wt[i]] + val[i])$ if $wt[i] \leq w$, else $dp[i-1][w]$

```
def knapsack(weights, values, W):
    n = len(weights)
    dp = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(W + 1):
            dp[i][w] = dp[i - 1][w]
            if weights[i - 1] <= w:
                dp[i][w] = max(dp[i][w],
                               dp[i - 1][w - weights[i - 1]] + values[i - 1])
    return dp[n][W]
```

Dry Run: $weights=[1,3,4,5]$, $values=[1,4,5,7]$, $W=7$

$i \backslash w$	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1 (w_1, v_1)	0	1	1	1	1	1	1	1
2 (w_3, v_4)	0	1	1	4	5	5	5	5
3 (w_4, v_5)	0	1	1	4	5	6	6	9
4 (w_5, v_7)	0	1	1	4	5	7	8	9

At $dp[4][7]$: skip item4 $\rightarrow dp[3][7]=9$; take item4 $\rightarrow dp[3][2]+7=1+7=8$; $\max(9,8)=9 \rightarrow$ **Answer: 9**

Time: $O(n \cdot W)$ | Space: $O(n \cdot W)$, optimizable to $O(W)$ using a 1D array iterated right-to-left.

4.2 Subset Sum

Problem: Given a set of numbers, determine if a subset sums exactly to target S .

Recurrence: $dp[i][s] = dp[i-1][s]$ OR $(s \geq \text{nums}[i-1] \text{ AND } dp[i-1][s-\text{nums}[i-1]])$

```
def subsetSum(nums, target):
    n = len(nums)
    dp = [[False] * (target + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        dp[i][0] = True
    for i in range(1, n + 1):
        for s in range(1, target + 1):
            dp[i][s] = dp[i - 1][s]
            if nums[i - 1] <= s:
                dp[i][s] = dp[i][s] or dp[i - 1][s - nums[i - 1]]
    return dp[n][target]
```

Dry Run: $nums = [3, 34, 4, 12, 5, 2]$, $target = 9$

$4 + 5 = 9$, or $3 + 4 + 2 = 9 \rightarrow$ both are valid subsets $\rightarrow$ **Answer: True**

Table logic: $dp[i][s]$ becomes True the moment either "exclude $\text{nums}[i-1]$ " ($dp[i-1][s]$) or "include $\text{nums}[i-1]$ " ($dp[i-1][s-\text{nums}[i-1]]$) is already True — once True for a column it propagates down.

4.3 Coin Change — Minimum Coins

Problem: Given coin denominations and an amount, find the fewest coins needed to make that amount (unlimited supply of each coin).

State: $dp[a]$ = minimum coins to make amount a .

Recurrence: $dp[a] = \min(dp[a - \text{coin}] + 1)$ for every $\text{coin} \leq a$; $dp[0] = 0$.

```
def coinChange(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for a in range(1, amount + 1):
        for coin in coins:
```

```

    if coin <= a:
        dp[a] = min(dp[a], dp[a - coin] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1

```

Dry Run: coins = [1, 2, 5], amount = 11

a	0	1	2	3	4	5	6	7	8	9	10	11
dp[a]	0	1	1	2	2	1	2	2	3	3	2	3

dp[11] = min(dp[10]+1, dp[9]+1, dp[6]+1) = min(3, 4, 3) = 3 → **Answer: 3** (5+5+1)

4.4 Coin Change — Count Ways

Problem: Count the number of distinct ways to make an amount using given coins (order doesn't matter, unlimited supply).

Recurrence: loop coins outer, amounts inner: dp[a] += dp[a - coin]; dp[0] = 1

```

def change(amount, coins):
    dp = [0] * (amount + 1)
    dp[0] = 1
    for coin in coins:
        for a in range(coin, amount + 1):
            dp[a] += dp[a - coin]
    return dp[amount]

```

Dry Run: coins = [1, 2, 5], amount = 5

After processing coin=1: dp = [1,1,1,1,1,1]

After processing coin=2: dp = [1,1,2,2,3,3]

After processing coin=5: dp = [1,1,2,2,3,4]

Answer: 4 ways — {1×5}, {1×3+2}, {1×1+2×2}, {5}

Key insight: coin-outer/amount-inner loop order counts *combinations* (order doesn't matter). Swapping loop order (amount-outer/coin-inner) counts *permutations* instead.

5. Pattern 4 — Longest Increasing Subsequence (LIS)

PATTERN: $dp[i]$ = best ending exactly at index i

Problem: Find the length of the longest strictly increasing subsequence in an array.

State: $dp[i]$ = length of the LIS ending at index i .

Recurrence: $dp[i] = \max(dp[j] + 1)$ for all $j < i$ where $nums[j] < nums[i]$; base $dp[i] = 1$.

```
def lengthOfLIS(nums):
    n = len(nums)
    dp = [1] * n
    for i in range(n):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)
```

Dry Run: $nums = [10, 9, 2, 5, 3, 7, 101, 18]$

index	0	1	2	3	4	5	6	7
$nums[i]$	10	9	2	5	3	7	101	18
$dp[i]$	1	1	1	2	2	3	4	4

$dp[3]=2$ (2,5); $dp[5]=3$ (2,5,7) or (2,3,7); $dp[6]=4$ (2,5,7,101) → **Answer: 4**

Time: $O(n^2)$ with this method | $O(n \log n)$ possible using binary search on a "tails" array.

5.1 $O(n \log n)$ Optimized LIS

```
import bisect
def lengthOfLIS_fast(nums):
    tails = []
    for x in nums:
        pos = bisect.bisect_left(tails, x)
        if pos == len(tails):
            tails.append(x)
        else:
            tails[pos] = x
    return len(tails)
```

Idea: `tails[k]` holds the smallest possible tail value of an increasing subsequence of length $k+1$. The length of `tails` at the end is the LIS length.

6. Pattern 5 — String DP

PATTERN: $dp[i][j]$ compares prefixes of two strings, or $i..j$ window of one string

6.1 Longest Common Subsequence (LCS)

Problem: Given two strings, find the length of their longest common subsequence.

State: $dp[i][j]$ = LCS length of $text1[0..i)$ and $text2[0..j)$.

Recurrence: if $text1[i-1]==text2[j-1]$: $dp[i][j] = dp[i-1][j-1] + 1$; else $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$

```
def longestCommonSubsequence(text1, text2):
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]
```

Dry Run: $text1 = "abcde"$, $text2 = "ace"$

	""	a	c	e
""	0	0	0	0
a	0	1	1	1
b	0	1	1	1
c	0	1	2	2
d	0	1	2	2
e	0	1	2	3

Match 'a', match 'c', match 'e' along the diagonal path → **Answer: 3** ("ace")

6.2 Edit Distance (Levenshtein)

Problem: Minimum operations (insert, delete, replace) to convert $word1$ into $word2$.

Recurrence: if $word1[i-1]==word2[j-1]$: $dp[i][j]=dp[i-1][j-1]$; else $dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$

```
def minDistance(word1, word2):
    m, n = len(word1), len(word2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if word1[i - 1] == word2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j],      # delete
                                   dp[i][j - 1],      # insert
                                   dp[i - 1][j - 1])   # replace
    return dp[m][n]
```

Dry Run: $word1 = "horse"$, $word2 = "ros"$

	""	r	o	s
""	0	1	2	3
h	1	1	2	3
o	2	2	1	2
r	3	2	2	2
s	4	3	3	2
e	5	4	4	3

horse → rorse (replace h→r) → rose (delete r) → ros (delete e) → **Answer: 3**

6.3 Longest Palindromic Substring

Problem: Find the longest substring of s that is a palindrome.

State: $dp[i][j]$ = True if $s[i..j]$ is a palindrome.

Recurrence: $dp[i][j] = (s[i]==s[j]) \text{ AND } (j-i<2 \text{ OR } dp[i+1][j-1])$; fill by increasing substring length.

```
def longestPalindrome(s):
    n = len(s)
    dp = [[False] * n for _ in range(n)]
    start, maxLength = 0, 1
    for i in range(n):
        dp[i][i] = True
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            if s[i] == s[j]:
                if length == 2 or dp[i + 1][j - 1]:
                    dp[i][j] = True
                    if length > maxLength:
                        start, maxLength = i, length
    return s[start:start + maxLength]
```

Dry Run: $s = \text{"babad"}$

length=1: all single chars True.

length=2: "ba" no, "ab" no, "ba" no, "ad" no.

length=3: check "bab" $\rightarrow s[0]==s[2]='b'$, $dp[1][1]=\text{True} \rightarrow dp[0][2]=\text{True}$ (len 3). Also "aba" $\rightarrow \text{True}$ (len 3).

Longest found: "bab" (first one found) $\rightarrow$ **Answer: "bab"** (or "aba", both length 3)

7. Pattern 6 — Interval / Partition DP

PATTERN: $dp[i][j]$ = best way to solve the sub-range $[i, j]$, split at some k

7.1 Matrix Chain Multiplication

Problem: Given dimensions of matrices to multiply in sequence, find the minimum number of scalar multiplications needed.

State: $dp[i][j]$ = min cost to multiply matrices $i..j$.

Recurrence: $dp[i][j]$ = min over k in $[i, j-1]$ of $dp[i][k] + dp[k+1][j] + p[i-1]*p[k]*p[j]$

```
def matrixChainOrder(p):
    n = len(p) - 1 # number of matrices
    dp = [[0] * (n + 1) for _ in range(n + 1)]
    for length in range(2, n + 1):
        for i in range(1, n - length + 2):
            j = i + length - 1
            dp[i][j] = float('inf')
            for k in range(i, j):
                cost = dp[i][k] + dp[k + 1][j] + p[i - 1] * p[k] * p[j]
                dp[i][j] = min(dp[i][j], cost)
    return dp[1][n]
```

Dry Run: $p = [40, 20, 30, 10, 30]$ (4 matrices)

Matrices: $A_1(40 \times 20)$, $A_2(20 \times 30)$, $A_3(30 \times 10)$, $A_4(10 \times 30)$

Split	Cost expression	Value
$(A_1)(A_2A_3A_4)$	$40 \cdot 20 \cdot 30 + \text{cost}(A_2A_3A_4)$	26000
$(A_1A_2)(A_3A_4)$	$40 \cdot 30 \cdot 10 (A_1A_2) + 20 \cdot 30 \cdot 10 + 30 \cdot 10 \cdot 30 (A_3A_4) + 40 \cdot 20 \cdot 10$	38000
$(A_1A_2A_3)(A_4)$	$\text{cost}(A_1A_2A_3) + 40 \cdot 10 \cdot 30$	26000

Best split evaluated recursively gives **Answer: 26000** scalar multiplications (parenthesization $((A_1(A_2A_3))A_4)$).

7.2 Burst Balloons (brief)

Problem: Burst balloons to maximize coins; bursting balloon i gives $nums[\text{left}] * nums[i] * nums[\text{right}]$ coins.

State: $dp[i][j]$ = max coins from bursting all balloons strictly between i and j (i, j act as boundaries, added as sentinels of value 1).

Recurrence: $dp[i][j]$ = max over k in (i, j) of $dp[i][k] + dp[k][j] + nums[i] * nums[k] * nums[j]$ (k is the *last* balloon burst in that range)

Key insight shared with Matrix Chain: think about which element is chosen *last/as the split point*, not first — this avoids re-deriving overlapping subranges.

8. Advanced — Bitmask DP

PATTERN: `dp[mask][i]` — mask tracks a subset of visited/used elements

8.1 Traveling Salesman Problem (TSP)

Problem: Given a cost matrix between n cities, find the minimum cost to visit every city exactly once and return to start.

State: `dp[mask][i]` = minimum cost to have visited exactly the set of cities in `mask`, ending at city i .

Recurrence: `dp[mask][i]` = min over j in mask, $j \neq i$ of `dp[mask without i][j] + cost[j][i]`

```
def tsp(cost):
    n = len(cost)
    FULL = (1 << n) - 1
    dp = [[float('inf')] * n for _ in range(1 << n)]
    dp[1][0] = 0 # start at city 0, mask = only city 0 visited

    for mask in range(1 << n):
        for u in range(n):
            if dp[mask][u] == float('inf') or not (mask & (1 << u)):
                continue
            for v in range(n):
                if mask & (1 << v):
                    continue
                nmask = mask | (1 << v)
                dp[nmask][v] = min(dp[nmask][v], dp[mask][u] + cost[u][v])

    return min(dp[FULL][i] + cost[i][0] for i in range(1, n))
```

Dry Run (concept): 4 cities, start at city 0

mask (binary)	cities visited	meaning
0001	{0}	<code>dp[0001][0] = 0</code> (start)
0011	{0,1}	<code>dp[0011][1] = cost[0][1]</code>
0111	{0,1,2}	<code>dp[0111][2] = dp[0011][1] + cost[1][2]</code>
1111	{0,1,2,3}	<code>dp[1111][3] = dp[0111][2] + cost[2][3]</code>

Final answer = min over ending city i of `dp[FULL][i] + cost[i][0]` (return to start).

Time: $O(2^n \cdot n^2)$ | Space: $O(2^n \cdot n)$ — feasible for n up to ~18-20.

9. Advanced — DP on Trees

PATTERN: dp[node] computed from dp[children], via post-order DFS

9.1 House Robber III

Problem: Houses form a binary tree. Rob houses to maximize money; cannot rob two directly connected houses (parent-child).

State: for each node, return a pair (rob_this, not_rob_this) = best sum if we do / don't rob this node.

Recurrence: rob_this = node.val + left.not_rob + right.not_rob; not_rob_this = max(left.rob, left.not_rob) + max(right.rob, right.not_rob)

```
class Node:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def rob(root):
    def dfs(node):
        if not node:
            return (0, 0) # (rob, not_rob)
        left = dfs(node.left)
        right = dfs(node.right)
        rob_this = node.val + left[1] + right[1]
        not_rob_this = max(left) + max(right)
        return (rob_this, not_rob_this)
    return max(dfs(root))
```

Dry Run: tree 3 → (2, 3) → (leaf, leaf, leaf, 1)

Tree: root=3, left child=2 (with right child=3), right child=3 (with right child=1)

Node	(rob, not_rob)
leaf nodes	(val, 0) e.g. (3,0) and (1,0)
node "2"	rob=2+0+0=2, not_rob=max(0,0)+max(3,0)=3 → (2,3)
node "3" (right subtree)	rob=3+0+0=3, not_rob=max(1,0)=1 → (3,1)
root "3"	rob=3+3+1=7, not_rob=max(2,3)+max(3,1)=3+3=6

max(7, 6) → **Answer: 7**

10. How to Recognize Which Pattern to Use

CHEAT SHEET

Clue in the problem	Likely pattern
"ways to climb / tile / reach step n"	1D Linear DP
"maximize/minimize picking non-adjacent items"	1D DP with skip (House Robber style)
"grid, only move right/down"	2D Grid DP
"items with weight & value, capacity limit"	0/1 Knapsack
"can we make sum X" / "partition into equal subsets"	Subset Sum
"coins, unlimited supply, min count or #ways"	Unbounded Knapsack (Coin Change)
"longest increasing / non-decreasing subsequence"	LIS pattern
"two strings, common subsequence/substring, edit ops"	2D String DP (LCS / Edit Distance)
"palindrome substring/partition"	Interval DP on one string
"order of merging/multiplying matters, min cost to combine"	Interval / Partition DP
"visit all n items exactly once, n is small ($\leq \sim 20$)"	Bitmask DP
"binary tree / tree, can't pick parent & child together"	DP on Trees

Final tip: Whenever you see "count the number of ways", "find min/max", or "is it possible to achieve X" combined with a choice at each step (include/exclude, take/skip, split here/there) — think DP. Always write the recurrence in plain English first, then base cases, then code it.